

Notebook: decsion_tree/decision_tree_from_scratch.ipynb

[--- MARKDOWN CELL ---]

Decision Tree from Scratch (ID3 / CART style)

This notebook implements a numeric-feature decision tree from scratch. Each cell defines one function. Both Gini impurity and Information Gain (entropy) are implemented and selectable via the `criterion` parameter. We use the Iris dataset and matplotlib for plotting.

[--- CODE CELL ---]

```
# Imports
import math
import random
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
%matplotlib inline
```

[--- CODE CELL ---]

```
def load_iris_dataset():
    """Load Iris dataset from sklearn and return X, y, feature_names, target_names
    """
    from sklearn.datasets import load_iris
    data = load_iris()
    X = data.data
    y = data.target
    return X, y, data.feature_names, data.target_names
```

[--- CODE CELL ---]

```
def train_test_split_from_scratch(X, y, test_size=0.3, seed=None):
    if seed is not None:
        random.seed(seed)
    idx = list(range(len(X)))
    random.shuffle(idx)
    split_at = int(len(X) * (1 - test_size))
    train_idx = idx[:split_at]
    test_idx = idx[split_at:]
    return X[train_idx], X[test_idx], [y[i] for i in train_idx], [y[i] for i in test_idx]
```

[--- CODE CELL ---]

```
def entropy(labels):
    """Compute entropy of a list of class labels.
    """
    n = len(labels)
    if n == 0:
        return 0.0
    counts = Counter(labels)
    ent = 0.0
    for c in counts.values():
        p = c / n
        ent -= p * math.log2(p) if p > 0 else 0
    return ent
```

[--- CODE CELL ---]

```
def gini_index(groups, classes):
    """Compute Gini index for groups.
    groups: list of label lists (e.g., [left_labels, right_labels])
    classes: list of all class labels
    """
    n_instances = sum([len(g) for g in groups])
```

```

if n_instances == 0:
    return 0.0
gini = 0.0
for group in groups:
    size = len(group)
    if size == 0:
        continue
    score = 0.0
    counts = Counter(group)
    for class_val in classes:
        p = counts.get(class_val, 0) / size
        score += p * p
    gini += (1.0 - score) * (size / n_instances)
return gini

```

[--- CODE CELL ---]

```

def best_split(X, y, criterion='entropy'):
    """Find the best feature and threshold to split on.
    criterion: 'entropy' or 'gini'
    Returns dict with index, threshold, groups (left_idx, right_idx), and score.
    """
    n_samples, n_features = X.shape
    best = {'index': None, 'threshold': None, 'score': -float('inf'), 'groups': None}
    classes = list(set(y))
    for feature_idx in range(n_features):
        # consider midpoints between sorted unique values
        values = sorted(set(X[:, feature_idx]))
        thresholds = [(values[i] + values[i+1]) / 2.0 for i in range(len(values)-1)]
        for thr in thresholds:
            left_idx = [i for i, row in enumerate(X) if row[feature_idx] <= thr]
            right_idx = [i for i, row in enumerate(X) if row[feature_idx] > thr]
            left_labels = [y[i] for i in left_idx]
            right_labels = [y[i] for i in right_idx]
            if criterion == 'entropy':
                # information gain: parent_entropy - weighted_child_entropy
                parent_ent = entropy(y)
                n = len(y)
                w_ent = 0.0
                for group in (left_labels, right_labels):
                    w_ent += (len(group)/n) * entropy(group) if len(group) > 0 else 0
                info_gain = parent_ent - w_ent
                score = info_gain
            else:
                # for gini we use negative gini as score (lower gini is better)
                gini = gini_index([left_labels, right_labels], classes)
                score = -gini
            if score > best['score'] and len(left_labels) > 0 and len(right_labels) > 0:
                best = {'index': feature_idx, 'threshold': thr, 'score': score, 'groups': (left_idx,
right_idx)}
    return best

```

[--- CODE CELL ---]

```

def to_terminal(y):
    """Create a terminal node value (majority class).
    """
    return Counter(y).most_common(1)[0][0]

```

[--- CODE CELL ---]

```

def split(node, X, y, max_depth, min_size, depth, criterion='entropy'):
    left_idx, right_idx = node['groups']
    del(node['groups'])
    # create left child

```

```

if not left_idx or not right_idx:
    # make terminal
    node['left'] = node['right'] = to_terminal([y[i] for i in left_idx + right_idx])
    return
# check for max depth
if depth >= max_depth:
    node['left'] = to_terminal([y[i] for i in left_idx])
    node['right'] = to_terminal([y[i] for i in right_idx])
    return
# process left child
if len(left_idx) <= min_size:
    node['left'] = to_terminal([y[i] for i in left_idx])
else:
    X_left = X[left_idx]
    y_left = [y[i] for i in left_idx]
    node_left = best_split(X_left, y_left, criterion=criterion)
    node['left'] = node_left
    split(node['left'], X_left, y_left, max_depth, min_size, depth+1, criterion)
:
,
: {
:
: [
1
,

```

[--- CODE CELL ---]

```

def predict_row(node, row):
    if isinstance(node, dict) and node.get('index') is not None:
        if row[node['index']] <= node['threshold']:
            return predict_row(node['left'], row)
        else:
            return predict_row(node['right'], row)
    else:
        # node is a terminal value
        return node

```

[--- CODE CELL ---]

```

def predict(node, X):
    return [predict_row(node, row) for row in X]

```

[--- CODE CELL ---]

```

def plot_two_features(X, y, feat_idx=(0,1), feature_names=None, title='Iris data'):
    plt.figure(figsize=(7,5))
    for label in np.unique(y):
        mask = np.array(y) == label
        plt.scatter(X[mask, feat_idx[0]], X[mask, feat_idx[1]], label=str(label), alpha=0.7)
    plt.xlabel(feature_names[feat_idx[0]] if feature_names else f'feat_{feat_idx[0]}')
    plt.ylabel(feature_names[feat_idx[1]] if feature_names else f'feat_{feat_idx[1]}')
    plt.title(title)
    plt.legend()
    plt.show()

```

[--- CODE CELL ---]

```

def simple_tree_text(node, depth=0):
    indent = ' ' * depth
    if isinstance(node, dict) and node.get('index') is not None:
        print(f"{indent}X[{node['index']}] <= {node['threshold']:.4f}?")
        simple_tree_text(node['left'], depth+1)
        simple_tree_text(node['right'], depth+1)

```

```

else:
    print(f"{indent}Predict -> {node}")

```

[--- CODE CELL ---]

```

# Runner: load data, build tree, predict and plot
X, y, feature_names, target_names = load_iris_dataset()
X_train, X_test, y_train, y_test = train_test_split_from_scratch(X, y, test_size=0.3, seed=2)
root = best_split(X_train, y_train, criterion='entropy')
# recursively split children using our split function (it expects groups in node)
split(root, X_train, y_train, max_depth=4, min_size=2, depth=1, criterion='entropy')
# show simple textual tree
simple_tree_text(root)
# predict and accuracy
from collections import Counter
y_pred = predict(root, X_test)
acc = sum(1 for a,b in zip(y_test, y_pred) if a==b)/len(y_test)
print(f'Accuracy on test: {acc:.4f}')
# plot first two features of training set
plot_two_features(X_train, y_train, feat_idx=(0,1), feature_names=feature_names, title='Train (first two features)')

```